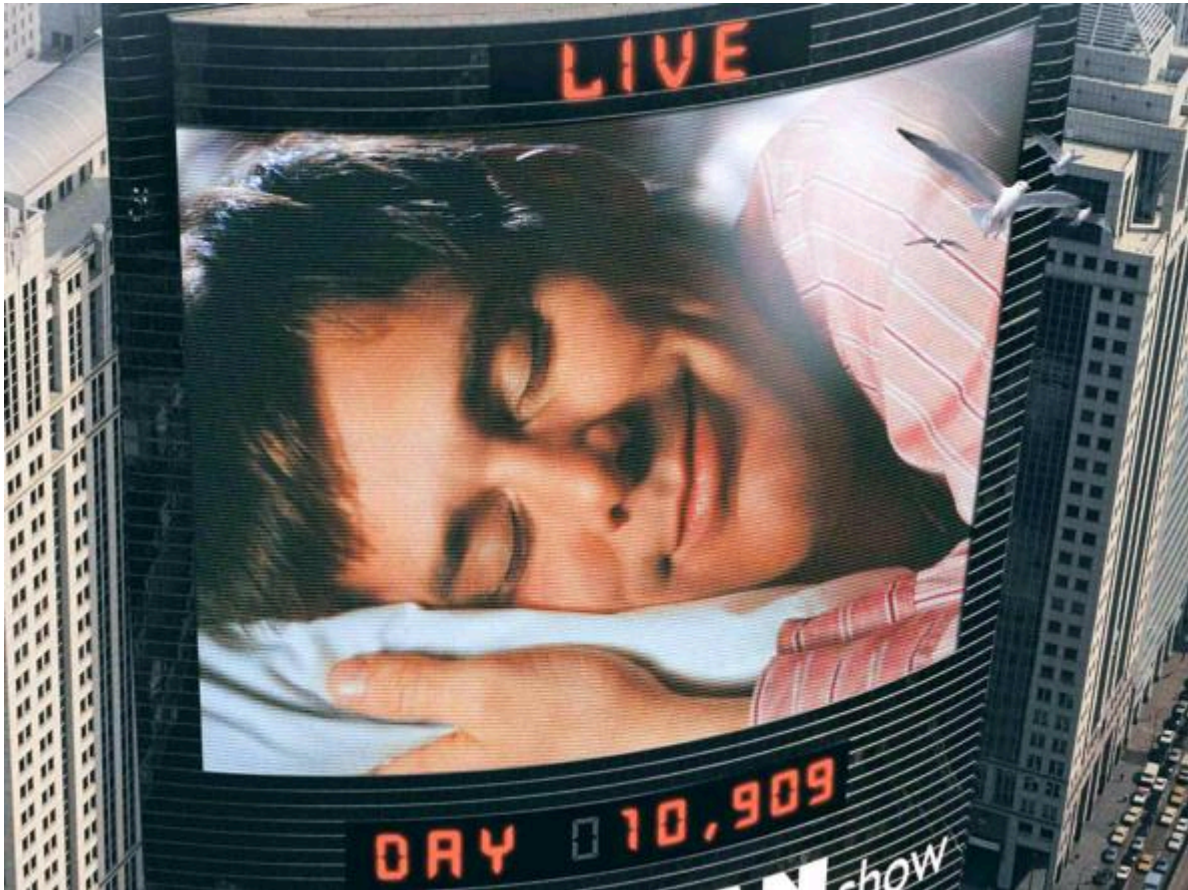


THE TRUMAN SHOW PROJECT

Andrea Meda : 20054214 / Asia Deiana : 20057114



Introduzione

Il progetto sviluppato ha come scopo l'osservazione di un agente intelligente, cioè autonomo e in grado di percepire l'ambiente in cui è stato inserito in modo da prendere decisioni e compiere azioni, in uno scenario pandemico, cioè un contesto simulato e controllato in cui viene introdotto il concetto di virus, con le relative meccaniche di propagazione e sviluppo.

Questo progetto nasce dalla necessità di osservare il comportamento di un singolo elemento intelligente, quando lo si inserisce in un ambiente controllato e simulato, affiancandolo ad elementi con comportamenti predefiniti, chiamati bot.

A questo proposito, data la particolare struttura del progetto, l'agente intelligente è stato ribattezzato con il nome di Truman, dato che solo a lui è data la possibilità di agire in modo indipendente, evolvendosi durante la simulazione, a differenza dei bot.

Per studiare l'agente sono state definite delle fasi pandemiche specifiche (illustrate di seguito), volte a sviluppare un apprendimento più graduale e constatabile, oltre che permettere l'osservazione di un ambiente variabile nel tempo, massimizzando così l'apprendimento dell'agente in base all'ambiente in cui è inserito.

Obiettivi del progetto

L'obiettivo principale del progetto è di analizzare il comportamento di un agente intelligente, posto in un ambiente controllato e costituito da elementi fissi (es. supermercato, ospedale ecc.) ed elementi mobili con cui interagire (i bot).

Dovranno essere osservate le decisioni prese dall'agente con l'evolversi della simulazione, i suoi movimenti, e il suo eventuale passaggio da uno stato di salute "non infetto" ad uno stato "infetto" durante le sue interazioni.

Potrà inoltre essere osservato lo sviluppo dell'ambiente simulato, come il progredire del virus nelle fasi iniziali e il graduale rallentamento dell'infezione durante le fasi finali (lockdown di tipo 3); sarà anche possibile osservare la comparsa di strutture apposite per la vaccinazione (centro vaccinale).

Scelte implementative

Di seguito sono illustrate le varie scelte implementative effettuate per lo sviluppo del progetto. Ciascuna scelta è stata raggruppata nella relativa sezione a cui appartiene.

I Bot :

Nella simulazione sono stati inclusi 149 bot. Con il termine bot ci si riferisce, in questo progetto, ad un'entità che simuli il più fedelmente possibile il comportamento ordinario di un cittadino in una situazione pandemica, ma senza poter compiere vere e proprie azioni autonome; in pratica, ogni comportamento è standardizzato e programmato in fase di implementazione. I bot, in questo progetto, svolgono una duplice funzione:

- 1) Ridurre il carico computazionale richiesto dall'applicazione, in quanto le risorse a disposizione per questo progetto non consentivano la gestione di più agenti intelligenti simultaneamente e per un tempo sufficiente per effettuare un'analisi dell'evoluzione del loro comportamento.
- 2) Rendono la simulazione più realistica e credibile, dato che, avendo comunque delle routine programmate, il loro comportamento nella simulazione li rende comunque un elemento da cui l'agente può apprendere ed evolversi di conseguenza.

Relativamente al punto due, i bot sono stati suddivisi in 3 gruppi principali; ogni bot verrà assegnato ad uno dei gruppi ad estrazione casuale ad ogni avvio della simulazione:

- 1) **Novax**: rappresentano il 22% della popolazione, con età compresa tra 22 e 55 anni; questi cittadini possono contrarre più facilmente forme più gravi del virus (cioè forme con una mortalità maggiore), restano infetti più a lungo rispetto agli altri cittadini

(180-360 ore contro le 144-280 standard), hanno una capacità di contagio molto più elevata ma una mortalità meno elevata.

- a) **Nota:** questi cittadini si ammalano maggiormente e di forme più gravi, e sono necessari per far progredire il contagio, ma sono più resistenti al virus, in modo da poter continuare a diffonderlo anche nelle fasi più avanzate della simulazione.
- 2) **Young** (Giovani): rappresentano il 30% della popolazione, con età compresa tra 18 e 35 anni. Questi cittadini contraggono in media forme di virus meno gravi (con mortalità minore), hanno un tasso di guarigione più elevato (in modo che anche l'agente abbia minore probabilità di morire), e una capacità di contagio ridotta.
- 3) **Normal:** rappresentano il 48% della popolazione, con età compresa tra 36 e 60 anni. Questi cittadini sono conformi ai parametri standard globali definiti nel progetto (SEVERE_PROB_BASE, DEATH_PROB_HOSPITAL, ecc.)

Questi gruppi sono necessari a diversificare le azioni svolte nella simulazione in un singolo giorno, in modo da non rendere la simulazione “piatta” e “finta”.

L'interazione dei bot con l'agente prevede la possibilità di essere contagiato, se quest'ultimo entra in contatto con un bot infetto; in pratica, l'interazione prevista nella simulazione è volta solo al progredimento del contagio, senza fornire dati o sviluppi sulle singole interazioni.

- **NOTA:** non sono stati inseriti cittadini anziani per mantenere il modello più semplice possibile e concentrarsi maggiormente sull'agente intelligente; inoltre, i parametri sopra riportati possono essere modificati a seconda dello scenario pandemico scelto per la simulazione.

Ogni cittadino appartenente ad uno dei tre gruppi indicati in precedenza viene poi suddiviso in altri 5 sottogruppi, effettuando un calcolo basato sull'ID del cittadino in questione e il gruppo di appartenenza. Questa ulteriore suddivisione è volta a rendere la simulazione più realistica, modificando lo schedule di ogni gruppo in modo che non tutti i cittadini di uno dei tre grandi gruppi svolgano simultaneamente la stessa operazione. In pratica, con questa soluzione, l'ambiente simulato viene reso visivamente più credibile e favorisce un'attività più ordinata e comprensibile, mascherando il più possibile il comportamento standardizzato dei bot.

Le fasi pandemiche :

Sempre per fornire maggiore dinamicità al progetto, sono state stabilite delle fasi pandemiche, volte a modificare il comportamento dei bot e l'ambiente in cui viene eseguita la simulazione, tramite l'aggiunta di luoghi specifici (il centro vaccinale); queste variazioni servono per fornire all'agente informazioni aggiuntive e un ambiente più “vivo” da cui poter apprendere, favorendo quindi lo sviluppo di comportamenti intelligenti volti alla sua sopravvivenza.

Nello specifico, sono state definite 3 fasi pandemiche:

- 1) **Fase 1:** Prima fase di lockdown, raggiungibile quando il numero di infetti raggiunge una soglia complessiva di 25 cittadini (o $\frac{1}{10}$ della popolazione). In questa fase, il parco resta accessibile ai cittadini (aperto), mentre il bar adotterà orari ridotti (18:00 –

21:00). Anche il supermercato, adotterà una fascia oraria ridotta (9:00 - 19:00). Viene inoltre adottato un orario di coprifuoco (alle ore 22:00).

- 2) **Fase 2** : Seconda fase di lockdown, raggiungibile quando il numero di infetti raggiunge una soglia complessiva di 50 cittadini (o $\frac{1}{3}$ della popolazione). In questa fase, il bar e il parco vengono chiusi (open_from_day viene incrementato ad un numero enorme, così questi luoghi rimarranno chiusi finché la fase pandemica non cambia), mentre il supermercato continuerà ad adottare gli orari della fase 1. L'orario di coprifuoco viene ulteriormente ridotto alle ore 21:00.
- 3) **Fase 3** : Questa è la fase più critica, raggiunta con un numero di contagiati minimo di 100 cittadini (o $\frac{2}{3}$ della popolazione). In questa terza fase, vengono imposti limiti molto più severi rispetto alle fasi precedenti, con bar e parco chiusi e l'utilizzo di orari molto ridotti per il supermercato (10:00 - 13:00). In questa fase, per mitigare il contagio, viene inserito nella mappa il centro vaccinale; quest'area sarà un punto in cui gli agenti potranno ricevere una vaccinazione che abbasserà la loro probabilità di subire un contagio e di riscontrare tipologie gravi (mortalità) del virus. Da questo momento, il centro vaccinale resterà visibile per tutto il resto della simulazione, e servirà come elemento base per la regressione del virus. Anche qui, l'orario di coprifuoco viene ridotto fino alle ore 20:00.

La simulazione include anche una fase iniziale (**Fase 0**) di pre-lockdown, in cui i cittadini hanno piena libertà e accesso alle strutture presenti nella città simulata, rispettando gli orari standard di ogni struttura:

- Supermercato: 9:00 - 20:00
- Bar: 18:00 -23:00
- Parco: orario continuato

L'agente (Truman) :

Per ridurre ulteriormente il costo computazionale del progetto, date le riscontrate difficoltà a rendere la simulazione lunga sufficientemente da effettuare un'analisi a causa delle risorse ridotte a disposizione, si è scelto di far compiere all'agente una decisione autonoma ogni 5 giorni simulati, facendogli quindi ripetere la stessa azione per i 5 giorni successivi alla decisione; al termine dei 5 gironi, osservano l'ambiente circostante e in base al suo stato di salute, l'agente effettuerà un'altra decisione.

Le decisioni dell'agente non si limiteranno unicamente ad un vincolo temporale, ma saranno anche definite in base a parametri stabiliti nel codice; in particolare, l'agente esegue chiamate a Gemini (nuove decisioni) se:

- La fase pandemica della simulazione cambia
- Lo stato di salute percepito cambia (non considerando lo stato E, che viene percepito dall'agente come S)
- Viene reso disponibile il centro vaccinale
- Sono passati un numero massimo di giorni dall'ultima spesa (visita al supermercato) dell'agente (l'agente non può stare a casa per tutta la simulazione o morirebbe di fame).

Nota: Questi vicoli non stabiliscono una "linea di condotta" fissa per l'agente. In ciascuno dei casi sopra riportati, l'agente è libero di compiere una decisione intelligente (es. se non fa la spesa da troppi giorni, ma è gravemente malato, sceglie comunque di restare a casa).

All'agente sono state fornite inoltre delle risorse da cui apprendere, tra cui documenti contenenti le regole applicate durante il contagio da Covid-19, per mitigare la diffusione pandemica; da questi documenti, l'agente apprenderà i comportamenti da adottare per evitare il contagio.

La personalità

La personalità è un parametro fondamentale che l'agente utilizzerà per definire le sue decisioni. A seconda dello stato attuale della simulazione, un agente deciderà quindi di variare la sua personalità (variando quindi dei suoi parametri), per poter quindi definire se eseguire un'azione piuttosto che un'altra (es. l'agente varia la sua personalità per incrementare la sua probabilità di uscire in un determinato momento della simulazione, in cui la propagazione del virus è contenuta, mentre sceglierà di abbassare la sua socialità quando la simulazione si trova in una fase pandemica avanzata).

Per definire i tratti caratteristici dell'agente e del suo gruppo di appartenenza si è tratta forte ispirazione dal celebre film "The Truman Show", in particolare riferendosi alle caratteristiche iniziali del personaggio: età giovane, in salute, resistente ai virus, tendente a seguire la routine nella fase iniziale (pre lockdown) e propenso ad un'evoluzione logica basata sull'osservazione dell'ambiente e delle sue caratteristiche.

Il centro vaccinale

Per ridurre la propagazione del contagio nelle fasi avanzate della simulazione, e cioè le fasi in cui il numero degli infetti ha raggiunto una quota critica (i 2/3 della popolazione complessiva, cioè ancora in vita) viene inserita una nuova struttura nella mappa, a cui i cittadini potranno accedere una sola volta per ricevere la vaccinazione.

Una vaccinazione sarà necessaria e sufficiente ad abbassare i parametri di esposizione di ciascun cittadino, in modo che quest'ultimo sia meno predisposto, da quel punto in poi, ad essere contagiato o a diffondere il contagio.

L'importanza di non vaccinarsi

Data la necessità di riprodurre correttamente una situazione realistica, basandosi su quello che si è già registrato osservando pandemie reali, insorge la necessità di prolungare ulteriormente la propagazione del virus, anche con l'aggiunta del centro vaccinale. Per questo scopo sono stati aggiunti i novax, il cui compito iniziale viene accentuato in questa fase; questi cittadini infatti non sono stati concepiti come diffusori del virus nelle prime fasi, in cui ogni cittadino aveva una probabilità medio-alta di contrarre il virus e questo poteva moltiplicarsi senza problemi. La fase 3 è, invece, il punto della simulazione in cui i novax diventano realmente vitali per un ulteriore sviluppo dell'agente interessato. Senza questa tipologia di cittadini, la fase 3 debellerebbe il virus in pochi giorni; è stato quindi necessario, in fase di progettazione, inserire dei bot che continuassero la diffusione del contagio, anche nelle fasi più critiche, e che fossero abbastanza resistenti da non morire appena contagiati, in modo da massimizzare l'effetto.

Grazie a questo tipo di cittadino, la fase 3 può essere prolungata ulteriormente e permette un'osservazione più precisa e prolungata del comportamento dell'agente in una fase critica. Questa necessità nasce anche dalla scelta di far compiere all'agente una vera decisione solo ogni 5 giorni; se quindi la fase 3 avesse interrotto il contagio dopo pochi giorni, si sarebbero osservate un numero insufficiente di decisioni prese dall'agente in questione. Adottando quindi la soluzione dei cittadini no vax, il contagio viene prolungato a sufficienza per poter visualizzare e valutare più decisioni distinte.

NOTA: è stato introdotto come parametro anche l'efficacia del vaccino, in modo da poter testare, variando i parametri di default del codice (contenuti nel file config.py), l'efficacia di diversi tipi di vaccino diversi.

Tecnologie utilizzate

Librerie utilizzate:

Libreria	A cosa serve
agno	Questo framework si occupa di: costruire il context ad ogni chiamata, registrare e gestire i tool (le 5 funzioni di sola lettura), gestire il ciclo ReAct (chiamare Gemini, possibile richiesta di un tool, esecuzione del tool, richiamare Gemini), collegarsi al database SQLite per storage/memoria, collegarsi a LanceDB per la knowledge, e forzare l'output nello schema DailyPolicy . Questo framework rende Gemini (un semplice modello che genera testo) utilizzabile come "agente" con capacità di agire, ricordare e consultare fonti esterne. In pratica, agno gestisce l'agente LLM (tool-use, memoria, knowledge/RAG, output strutturato).
google-genai	Libreria che permette a Python di parlare con Gemini (tramite chiamate API), connettendo il codice verso l'API di Gemini. Agno usa questa libreria per fare le vere chiamate HTTP verso i server di Google: mandare il prompt, ricevere la risposta JSON, gestire l'autenticazione con la chiave API. Senza questa libreria, Agno non avrebbe modo di parlare realmente con Gemini.
python-dotenv	Risolve un problema di sicurezza molto comune: la chiave API di Gemini (una stringa segreta utilizzata per identificare e autorizzare le chiamate) non deve mai stare scritta direttamente nel codice ; questo perchè, se il codice venisse pubblicato (es. su GitHub), chiunque potrebbe rubarla e usarla. La libreria python-dotenv legge un file separato chiamato .env (confinato solo nel computer del programmatore, ed escluso da Git) contenente righe del tipo GEMINI_API_KEY=abc123... ; queste righe vengono rese disponibili al codice Python come variabile d'ambiente (la chiave viene letta da un file separato, senza doverla scrivere nel codice, aumentando la sicurezza).
pydantic	È il validatore di dati utilizzato all'interno del codice. Definisce la struttura esatta di DailyPolicy (i 5 campi con i loro tipi e vincoli, es. sociability_today deve essere un numero tra 0 e 1). Quando Gemini restituisce il file JSON, Pydantic controlla che ogni campo sia presente, che sia del tipo giusto, e dentro i range permessi; se qualche elemento non è conforme ai parametri desiderati, solleva un

	errore invece di lasciar passare un dato corrotto o incompleto nel resto della simulazione. In pratica, questa libreria trasforma il testo generato dal modello in un dato strutturato e utilizzabile in sicurezza dal codice.
matplotlib.	Libreria utilizzata per la generazione dei grafici (plot_results , plot_ilm_behavior) e l'animazione della città (animate_simulation) in <code>visualization.py</code> . Serve per monitorare visivamente l'andamento dell'epidemia e il comportamento dell'agente nella simulazione, oltre che all'evoluzione dell'ambiente simulato.
numpy	Libreria usata per due scopi specifici: 1) in <code>visualization.py</code> per costruire la griglia di colori che rappresenta la mappa della città (build_city_background, una matrice 2D dove ogni cella ha un colore in base a cosa rappresenta), 2) generare sequenze numeriche (array di ore, di giorni) usate come assi nei grafici. Questa libreria è particolarmente utile quando si vogliono manipolare grandi quantità di numeri in modo veloce.
Pillow	Libreria usata per la manipolazione di immagini in Python. Dato che matplotlib da solo può creare un'animazione, ma per salvarla su disco come file GIF ha bisogno di un "writer" che sappia scrivere quel formato (matplotlib non può generare un file GIF da solo). Grazie a Pillow, tramite la chiamata di ani.save(filename, writer="pillow", ...) , l'animazione può essere convertita in un formato GIF: - ani: animazione già salvata in memoria da matplotlib - .save(): operazione per il salvataggio sul disco - writer="pillow": operazione che utilizza Pillow come writer, per scrivere il file GIF. In pratica, Pillow ricopre il ruolo di "traduttore" tra l'animazione disegnata da matplotlib e il file GIF finale, salvato sul disco. Senza Pillow, matplotlib non saprebbe come esportare l'animazione in un file GIF vero e proprio.
lancedb	È il vector database vero e proprio , cioè la tecnologia che fisicamente salva su disco gli embedding dei documenti (cioè i vettori numerici). Quando Agno utilizza LanceDB per la knowledge, quest'ultimo non reimplementa un database vettoriale da zero, ma si appoggia a questa libreria tramite un wrapper (codice intermedio usato per far comunicare due tecnologie diverse): (agno.vectordb.lancedb.LanceDb). Agno comunica con il wrapper usando i suoi comandi standard (es. "cerca documenti simili a X"), e il wrapper si occupa di tradurre quella richiesta nelle chiamate specifiche che LanceDB è in grado di capire, e di restituire i risultati in un formato comprensibile per Agno.

Addestramento dell'agente

Contex (cosa vede il modello a ogni run)

Ogni volta che dev'essere effettuata una nuova decisione, l'agente non riceve l'intero stato della simulazione, ma uno "snapshot" sintetico, costruito appositamente ad ogni nuova richiesta da sottoporre. In pratica, viene deciso manualmente, di volta in volta, cosa viene inserito nel prompt. In questo snapshot sono contenute le seguenti informazioni:

- **Informazioni sullo scenario:** giorno e ora correnti, scenario attivo (quali regole sono state adottate per lo scenario corrente), fase di lockdown
- **Informazioni personali sull'agente:** dati anagrafici e di personalità di Truman
- **Stato di salute dell'agente:** lo stato di salute di Truman e se risulta vaccinato o no
- **Azione del virus nell'ambiente e conseguenze:** tasso di infezione in città (che indica quanti cittadini sono infetti attualmente), e livello di occupazione dell'ospedale.
- **Informazioni sugli orari delle strutture:** orari di coprifuoco per ogni struttura
- **Dati di storage a breve termine:** un riassunto sintetico di cosa è successo il giorno prima
- **Dati di addestramento:** un estratto del documento normativo relativo alla fase corrente.

L'ultimo punto si basa sulla necessità di fornire a Truman un estratto del documento di indicazioni relativo alla fase di lockdown in corso, così che l'agente riceva anche un riferimento normativo coerente con il momento della simulazione. In pratica, si vanno a fornire all'agente le informazioni documentate relative alle pratiche da adottare relative alla fase pandemica in cui si trova (es. se l'agente si trova nella prima fase pandemica, verranno forniti i documenti inerenti ai nuovi comportamenti da adottare in questa fase).

- **Nota:** non vengono fornite tutte le informazioni e normative a disposizione per l'addestramento, dato che, trattandosi di un contesto reale, l'agente non potrebbe sapere in anticipo le nuove normative che verranno adottate in fasi più critiche della simulazione, perché questo causerebbe un comportamento anticipatorio e non inerente ad un contesto reale.

Relativamente allo stato di salute dell'agente, viene effettuata una scelta volta ad aumentare il realismo della simulazione. In particolare, ci si focalizza sul fatto che, sebbene durante la simulazione si conosca lo stato di salute preciso dell'agente, che è relativo al modello a stati del tipo SEIR + D:

- **S** : Sano ma con possibilità di infettarsi
- **E** : Esposto/asintomatico (il cittadino ha contratto il virus, ma quest'ultimo risulta in uno stato di incubazione, non presentando ancora sintomi evidenti)
- **I** : infetto (il cittadino presenta sintomi)
- **R**: rimosso (cittadino guarito, quindi tornato allo stato S).
- **D** : deceduto

Durante la simulazione, quando dev'essere effettuata una decisione (quindi quando si deve costruire lo snapshot), se lo stato di salute dell'agente (Truman) risulta essere E, questo verrà sovrascritto come S per prendere la decisione successiva. Questo particolare conferisce più realismo alla simulazione, infatti un cittadino asintomatico non può sapere in autonomia di aver contratto il virus, quindi questo parametro non può influire sulle sue decisioni.

In pratica, se lo stato reale è "E", nel context che arriva all'LLM, Truman viene descritto come ancora "S" (sano), perché nella realtà, una persona esposta ma asintomatica non sa di essere malata, e non ha modo di percepirla. Questo permette all'agente di prendere decisioni basandosi sulle stesse informazioni imperfette che avrebbe a disposizione un cittadino reale.

Nella simulazione viene comunque tenuto traccia dello **stato reale** dell'agente (Truman) con precisione assoluta; se l'agente è nello stato E, il sistema sa che è già esposto al virus, e continua a simularne correttamente la progressione "dietro le quinte" (l'agente potrebbe contagiare altri cittadini durante le sue attività esterne e passerà allo stato "I" dopo un lasso di tempo). In pratica, solo lo stato comunicato all'agente viene filtrato al fine di prendere decisioni il più possibile realistiche.

Tools (l'agente agisce nel mondo)

All'agente sono stati forniti cinque tool read-only (di sola lettura), che quest'ultimo può decidere autonomamente di richiamare durante il proprio ragionamento qualora ritenga di aver bisogno di conferme aggiuntive prima di rispondere. I 5 tools sono:

- stato personale (salute, età, vaccinazione)
- tasso di infezione in città e fase di lockdown attuale
- occupazione dell'ospedale
- luoghi pubblici aperti in quel momento (bar, supermercato, parco, ed eventualmente il centro vaccinale, se già comparso nella simulazione) + orario coprifuoco
- esito della giornata precedente

In pratica, l'agente (Truman) non può mai modificare lo stato della simulazione tramite tool, ma può solo leggerlo. I tools dell'agente servono quindi a fornire un accesso più mirato e verificabile ai dati, dato che lo snapshot iniziale nel context potrebbe non bastare o potrebbe essere leggermente disallineato nel tempo (a causa di valori variabili rapidamente, come lo stato di occupazione dell'ospedale). L'agente, durante il proprio ragionamento (il ciclo ReAct), può quindi decidere autonomamente di richiamare uno o più di questi tool se sente il bisogno di una conferma più aggiornata o più mirata, invece di fidarsi ciecamente del riassunto ricevuto in partenza.

La scelta di utilizzare solo tool read-only è basata sulla natura del progetto, in cui la decisione finale su cosa succede realmente nella simulazione deve restare sempre sotto il controllo del codice deterministico, non dell'LLM. Se un tool potesse riscrivere dei parametri della simulazione (es. `set_infection_rate()`), un errore del modello potrebbe corrompere lo stato della simulazione in modo imprevedibile e difficile da tracciare. Con tool solo in lettura, l'LLM può consultare la realtà simulata per decidere meglio, ma non può alterarla.

Reasoning (l'agente ragiona prima/durante le azioni)

Il modello riceve, all'interno del system prompt, due tipi di informazioni:

- 1) **Vincoli "hard"**: regole che devono sempre essere rispettate, per esempio:
 - a) "Se un cittadino risulta infetto, evita luoghi affollati"
 - b) "Se l'ospedale è saturo e il caso è grave, resta a casa".
- **Nota**: questi sono vincoli comunicati **nel prompt come istruzioni**, quindi il modello è invitato fortemente a rispettarli, ma i vincoli davvero "hard" (cioè che non possono mai essere violati in nessun caso) vengono comunque applicati anche a livello di codice deterministico dopo la decisione dell'LLM, come doppia sicurezza (evitando quindi che le decisioni prese escano dai limiti stabiliti in fase di progettazione).

- 2) **Indicazioni di personalità/ruolo:** Truman viene descritto come un cittadino giovane e non contrario al vaccino (non rientra nella categoria no vax). Questi elementi servono a dare coerenza comportamentale; servono quindi a caratterizzare il modello, evitando che quest'ultimo possa generare decisioni plausibili ma incoerenti da un giorno all'altro (un giorno molto cauto, il giorno dopo spericolato senza motivo).

Partendo da questi elementi:

Vincoli hard + personalità + dati dei tool consultati + knowledge recuperata

il modello è in grado di **ragionare autonomamente** (in questo contesto entra in gioco il ciclo ReAct: il modello può eseguire più operazioni di "pensiero → possibile chiamata di un tool → rivalutazione" prima di arrivare a una conclusione).

Utilità del ReAct

Senza l'impiego del ReAct, l'LLM dovrebbe rispondere basandosi solo sullo snapshot iniziale, senza possibilità di verificare le informazioni variabili. Con ReAct, il modello è in grado di chiamare il tool specifico, riceve il dato vero e aggiornato, e **solo dopo** formula la decisione finale. Questo approccio **permette al modello di raccogliere informazioni mancanti o verificare dati incerti prima di rispondere**, invece di basarsi solo sullo snapshot di partenza.

Alla fine, il modello è vincolato a restituire la propria decisione in un formato strutturato, chiamato **DailyPolicy**, e validato, tramite **Pydantic**; questo garantisce che ogni risposta sia sempre utilizzabile all'interno della simulazione, evitando output ambigui o incompleti.

In pratica, il ragionamento del modello può essere libero e articolato internamente, ma **l'output finale deve sempre avere una forma prevedibile e validata**, altrimenti il resto della simulazione (scritta in Python) non saprebbe come interpretarlo.

Knowledge (consulta fonti esterne)

Oltre ai tool, l'agente ha accesso a una base di conoscenza esterna in stile RAG (generazione potenziata da recupero dati), costruita indicizzando alcuni documenti markdown:

- indicazioni generali per il cittadino
- protocollo della città
- fonti informative sul Covid-19

Questi dati sono contenuti in un LanceDB (il vector database), con i testi trasformati in vettori numerici tramite un embedder di Gemini e recuperati semanticamente dall'agente quando ritiene utile approfondire un argomento prima di decidere.

Embedding

Processo che trasforma un testo in un vettore (lista di numeri) che rappresenta **il suo significato** in uno spazio matematico, dove testi con significato simile finiscono "vicini" tra loro, anche se utilizzano parole diverse. In questo modo, quando l'agente cerca informazioni, il sistema trasforma la sua domanda in un vettore e trova i documenti con vettori più vicini (cioè **semanticamente pertinenti con la domanda posta**), non solo quelli che contengono le stesse parole esatte. In questo modo viene eseguita una ricerca "intelligente" e non rigidamente testuale su un ampio insieme di documenti.

Ai documenti base iniziali (knowledge base) forniti all'agente vengono aggiunti progressivamente dei documenti specifici per ogni fase di lockdown. Questi documenti verranno aggiunti **solo quando la specifica fase di lockdown viene effettivamente raggiunta** nella simulazione. Questo, come già detto, serve a evitare che Truman abbia un accesso anticipato a ordinanze non ancora emanate, mantenendo coerenza temporale con la narrazione della simulazione.

Storage / Sessions (ricorda la conversazione) + Memory

Per dare continuità al comportamento di Truman tra una decisione e l'altra, le sue interazioni vengono salvate in un **database SQLite dedicato**, associato a un **identificativo di sessione fisso** per lui. In pratica, ogni interazione dell'agente con Gemini viene taggata sempre con lo stesso "session_id", in modo che il sistema sappia sempre che tutte quelle righe nel database appartengono allo stesso cittadino osservato nel tempo, e non le confonda con quelle di altri agenti (prevedendo che il sistema, in futuro, possa ospitare più agenti).

Le **ultime sei chiamate** vengono incluse automaticamente nel contesto delle richieste successive, per dare continuità comportamentale a breve termine. In pratica, per evitare che l'agente prenda decisioni incoerenti con il comportamento adottato nei giorni precedenti, vengono utilizzate le informazioni salvate sulle decisioni prese negli ultimi 6 giorni (considerando che l'agente interagisce con gemini per elaborare una vera decisione solo una volta ogni 5 gironi, come visto in precedenza).

A questo si affianca un **meccanismo di apprendimento più a lungo termine (Memory)**, costituiti da un profilo e una memoria dell'agente, che si costruiscono progressivamente nel tempo e vengono anch'essi aggiunti al context delle chiamate future, ma **non si limitano agli ultimi 6 giorni trascorsi**, ma restano disponibili anche molto più avanti nella simulazione, in forma sintetica (viene quindi costruita una memoria di Truman nel tempo, che verrà utilizzata come contesto per le chiamate future).

Caratteristiche dell'agente

L'agente Truman non usa l'LLM fin dall'inizio, quindi non prende decisioni autonome fin dall'inizio della simulazione (Attivazione progressiva).

Fino al primo lockdown, Truman segue lo stesso codice scriptato degli altri bot del suo gruppo (i giovani); in pratica, in questa prima fase della simulazione, l'agente è indistinguibile dai bot, in termini di comportamento osservabile. Solo dalla prima fase di lockdown in poi l'agente inizierà a interpellare Gemini per decidere autonomamente la propria policy giornaliera, prendendo quindi decisioni intelligenti non dettate dal codice utilizzato dagli altri bot.

La richiesta di una nuova decisione all'LLM non avviene a ogni giorno di simulazione (Refresh selettivo della decisione), ma solo quando è realmente necessario, cioè quando:

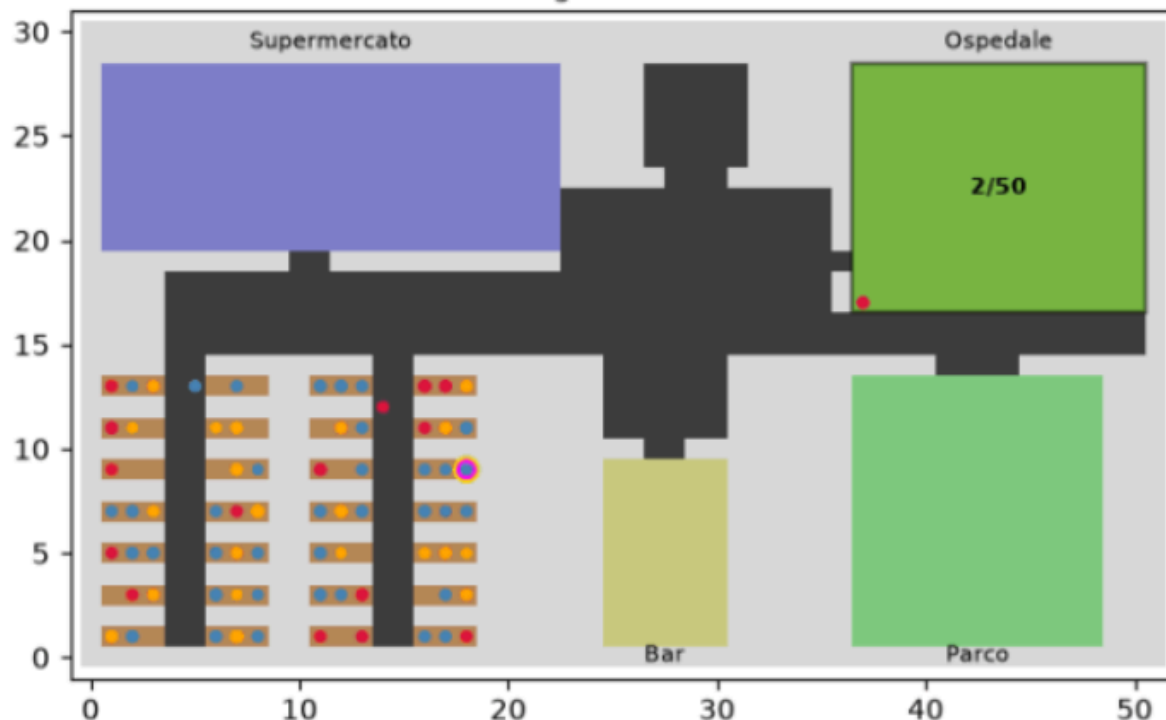
- cambia lo stato di salute percepito (es. comparsa dei sintomi).
 - **Nota:** la percezione subisce l'applicazione del filtro "E→S" visto sopra.
- cambia la fase di lockdown
- il centro vaccinale diventa disponibile per la prima volta
- è passato un certo numero di giorni dall'ultima decisione presa, in particolare nel progetto sono stati stabiliti 5 giorni tra una decisione e l'altra.

Negli altri casi, si riutilizza la policy del giorno precedente, permettendo di risparmiare chiamate API **senza risultare comportamentalmente statico**.

Gestione dei fallimenti:

Nel caso in cui Gemini non risponda (errore di rete, di quota, ecc.), è previsto un **retry con attesa crescente** (backoff), e in caso di fallimento persistente una **policy di fallback prudente**, composta da bassa socialità, basso rischio, destinazione "casa" (in pratica, l'agente, in assenza di risposta da Gemini, adotta un comportamento prudente, tendendo a restare nella sua abitazione). Questo meccanismo è pensato per evitare che un problema tecnico esterno possa mandare in crash l'intera simulazione.

La Mappa



Nell'immagine sopra riportata è possibile visualizzare gli elementi principali che caratterizzano l'ambiente simulato in cui l'agente è inserito.

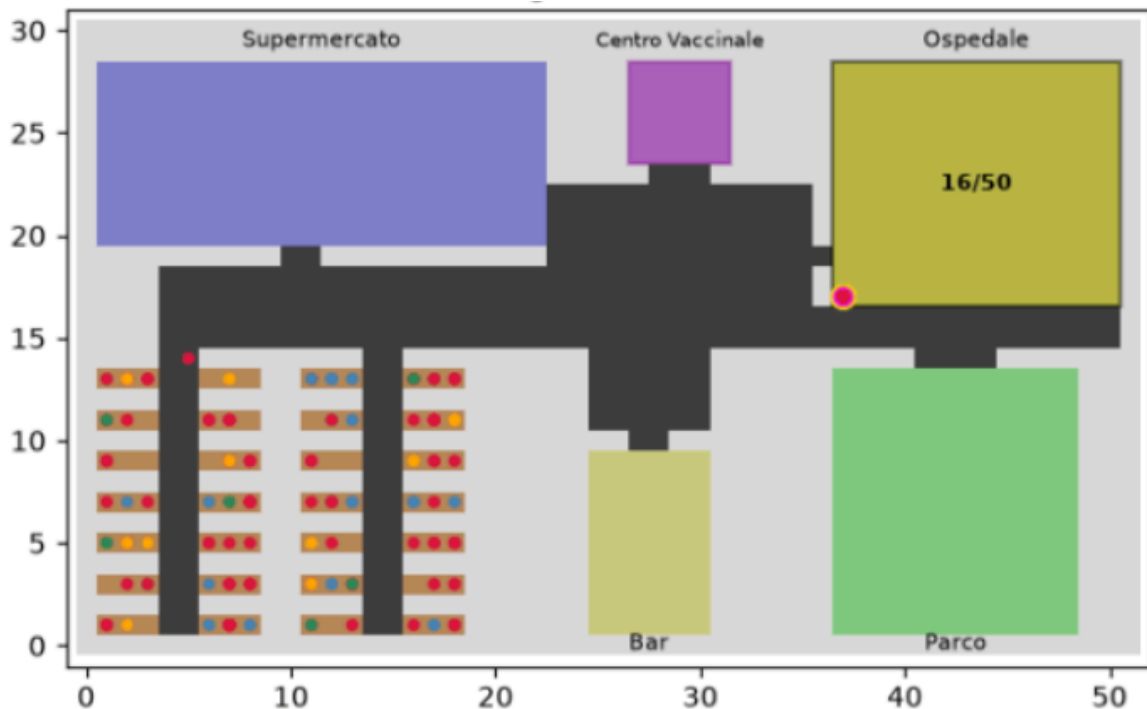
In questo ambiente possono essere evidenziati diversi elementi:

- **Agente (Truman)** : differenziato dagli altri elementi (cittadini/bot) grazie ad una caratterizzazione grafica più marcata, con contorni viola e gialli per renderlo più appariscente rispetto agli altri bot, e un colore centrale azzurro per ricordare la categoria di cittadini di cui fa parte.
- **Supermercato**: prima area di assembramento definita nella simulazione; permette a più cittadini di accedervi nello stesso momento, e funge da promotore per il contagio.
- **Bar**: seconda zona di assembramento. A differenza del supermercato, questa zona è puramente ricreativa e favorisce ulteriormente l'interazione, e quindi il contagio, tra i vari cittadini; a questo proposito, come si potrà osservare dalla fase 1 di lockdown, questo punto verrà dapprima gestito con una riduzione dell'orario di accesso da parte dei cittadini, e successivamente con una totale chiusura per verificare come le aree di assembramento abbiano favorito il contagio nelle fasi iniziali della simulazione.
- **Parco**: Ulteriore punto di assembramento, con lo stesso scopo del bar, ma con limitazioni restrittive nelle prime fasi di lockdown. Questo punto resterà aperto per più tempo, anche durante i lockdown di tipo 1, alimentando in modo più massivo il contagio, in particolare tra i cittadini no vax.
- **Ospedale**: Struttura adibita alla cura dei cittadini infetti (un cittadino malato si recherà all'ospedale per essere curato). Per la corretta simulazione di un ambiente reale e la necessità di protrarre la simulazione (e quindi la durata della pandemia) almeno fino al giorno 20, all'ospedale è stata data una capienza ridotta, in modo che

non tutti i cittadini contagiati possano recarsi per essere guariti; la capienza e il riempimento della struttura sono visualizzabili graficamente durante la presentazione.

- **Nota:** un cittadino infetto si recherà all'ospedale, indipendentemente dalla sua capienza; se quindi un ospedale non può ospitare il cittadino infetto, la sua mobilità verso la struttura costituirà un possibile fattore di rischio infettivo per gli altri cittadini che ne vengono a contatto lungo il tragitto.

La mappa nella fase di lockdown 3



Come già descritto in precedenza, la fase 3 costituisce il momento di massimo rischio per la simulazione, con una propagazione del virus quasi assoluta sulla popolazione analizzata. Per evitare che il virus si diffonda fino a tutti i cittadini, oltre all'applicazione di regole più severe, viene introdotta una nuova struttura:

- **Centro vaccinale:** struttura adibita alla vaccinazione dei cittadini, seguendo dei vincoli particolari:
 - 1) Ogni cittadino può ricevere al massimo una dose di vaccino (quindi ogni cittadino può vaccinarsi una sola volta). La vaccinazione garantirà un abbassamento del rischio di contagio e dell'insorgere di forme gravi del virus.
 - 2) I cittadini infetti non potranno vaccinarsi fino a dopo la guarigione. Questo vincolo è basato sulle caratteristiche del vaccino contro il Covid-19, che non poteva essere somministrato a cittadini infetti dato che non poteva curare il paziente, oltre al fatto che la presenza di un soggetto infetto nel centro vaccinale causerebbe un ulteriore focolaio.

AI di supporto utilizzate

Per lo sviluppo di questo progetto sono state utilizzate le seguenti AI:

- **Claude**: per la ricerca, l'approfondimento di concetti e risorse, e il supporto allo sviluppo del codice.
- **Gemini**: per la ricerca e la gestione delle chiamate API da parte dell'agente per effettuare le decisioni all'interno della simulazione.
- **Cursor**: per il supporto alla programmazione e la risoluzione di problemi e bug complessi.

Conclusione

Lo sviluppo del progetto ha evidenziato come l'agente, posto in uno scenario pandemico simulato ma il più possibile realistico, avendo a disposizione le nozioni necessarie alla gestione delle varie fasi pandemiche durante la simulazione, senza però ricevere nessun tipo di anticipazione o istruzione preventiva sulle fasi successive, sia stato in grado di apprendere, il più delle volte, i comportamenti più adeguati per poter preservare al meglio la propria salute e garantire la propria sopravvivenza.

Durante la simulazione sono stati adottati anche scenari differenti, basati su due tipologie di virus ben più aggressive (es. peste), per verificare come potesse variare il comportamento dell'agente in questi contesti. Da questi test aggressivi è emerso un dato interessante; certi virus non permettevano in nessun modo e con nessun comportamento all'agente di sopravvivere, valorizzando l'importanza di un adattamento dei regolamenti già noti sulla base dei virus coinvolti. In pratica, la mitigazione di un contagio e la sopravvivenza di un agente non è sempre garantita, anche se quest'ultimo rispetta totalmente le regole che gli sono state imposte, se queste regole non vengono adattate a tipi di virus diversi.

Questo progetto vuole verificare come scenari simili possano svilupparsi, evidenziando quali comportamenti possano favorire una proliferazione del contagio e quali invece possano mitigare il rischio in tempi brevi. Viene inoltre evidenziato l'enorme impatto provocato dai singoli cittadini no vax all'interno della simulazione.

Si evidenzia che, durante le simulazioni, l'agente intelligente ha superato, nella maggior parte dei casi, i giorni previsti per l'analisi simulata, sottolineando come il suo apprendimento, basato su informazioni reali (in particolare relative alla pandemia da Covid-19) sia progredito correttamente per preservare la sua sopravvivenza.

Nota finale:

A causa delle risorse limitate, questo progetto si è focalizzato unicamente sull'analisi di un singolo individuo in un contesto simulato; ulteriori sviluppi potrebbero portare ad un'analisi più completa e dettagliata, basata su gruppi di agenti, implementando interazioni più complesse e un ambiente più dinamico, dando la possibilità di analizzare come agenti con caratteristiche diverse possano ragionare in questo ambiente, coordinarsi nelle scelte effettuate o mostrare comportamenti riluttanti verso lo sviluppo del virus.